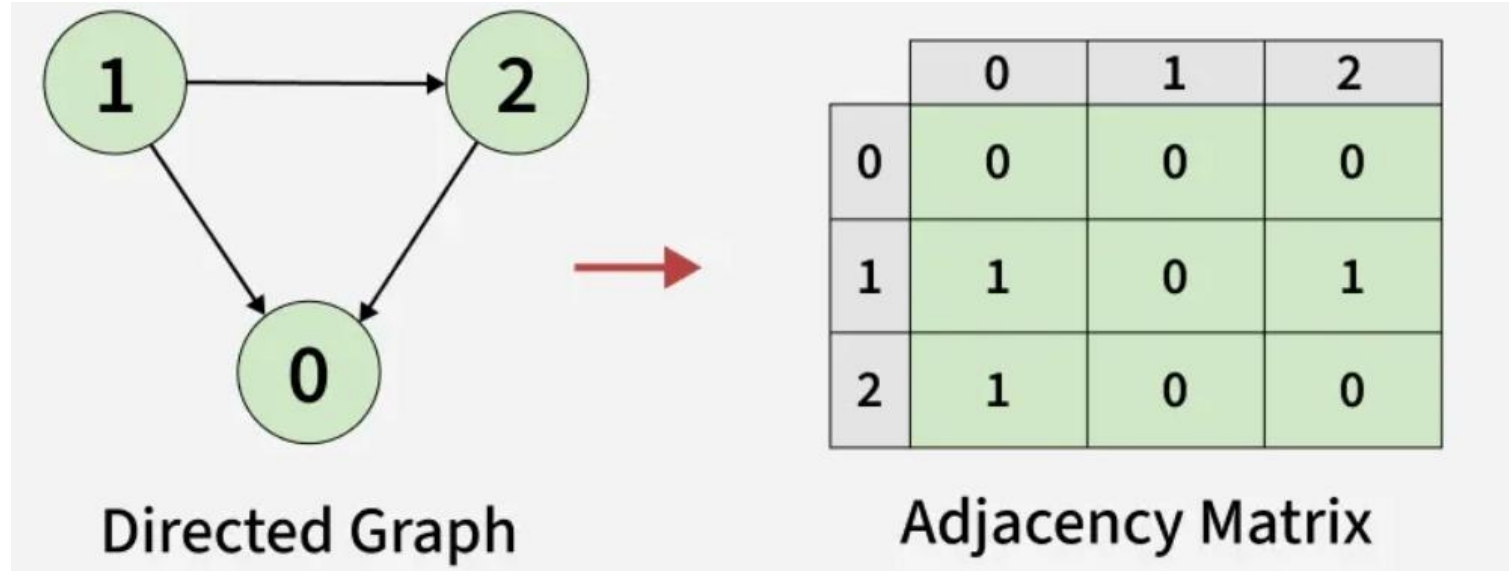


Graph Algorithms

Analysis and Design of Algorithms



Directed Graph as Adjacency Matrix



Representation of Directed Graph as Adjacency Matrix

- Function: `createGraph(V, edges)`
- Input:
 - V = Number of vertices
 - edges = List of directed edges $[u, v]$
- Output:

Adjacency matrix representing the graph

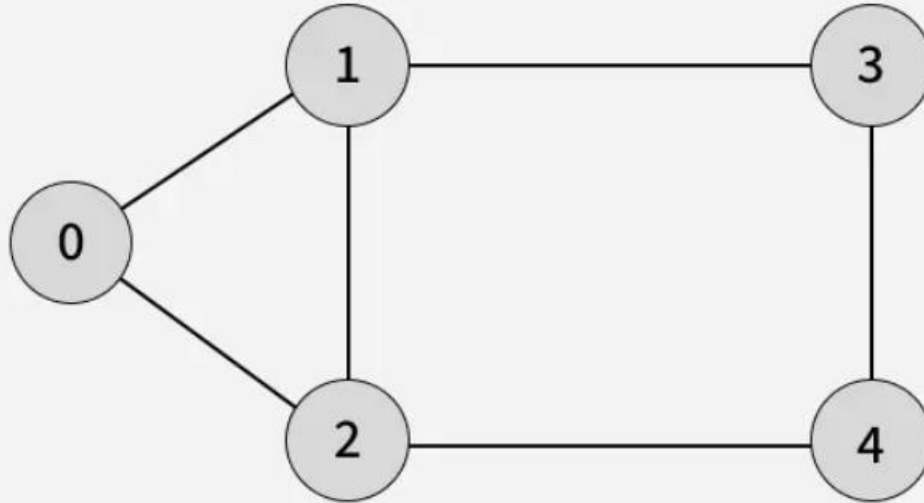


Trace Edge Insertions

Edge	Action	Matrix after update
[1,0]	<code>mat[1][0]=1</code>	0 0 0 1 0 0 0 0
[2,0]	<code>mat[2][0]=1</code>	0 0 0 1 0 0 1 0
[1,2]	<code>mat[1][2]=1</code>	0 0 0 1 0 1 1 0



Breadth First Search



Breadth First Search

Function: `bfs(adj)`

Input: Adjacency list `adj`

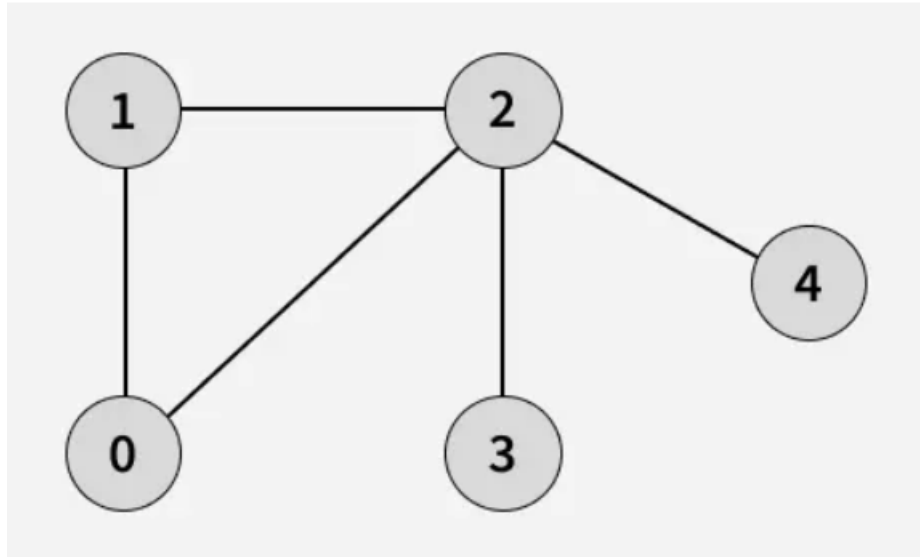
Output: BFS traversal starting from node 0

Graph Representation: Adjacency list using basic lists

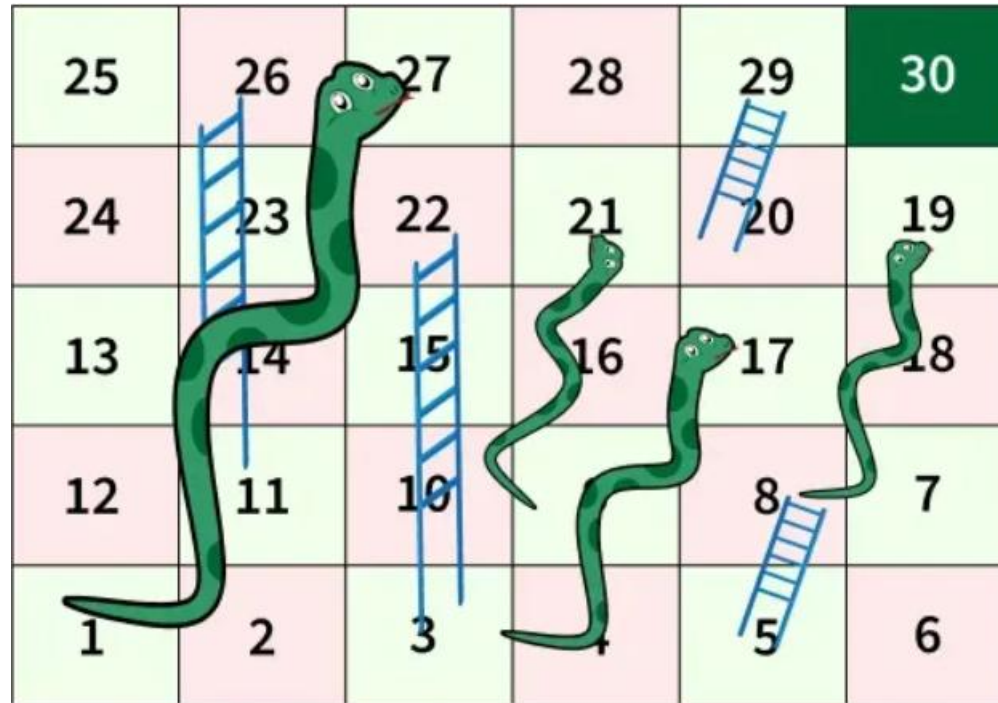


Depth First Search

Input: $\text{adj}[\text{[]}] = [[1, 2], [0, 2], [0, 1, 3, 4], [2], [2]]$



Example: Snake and Ladder problem



Example: Snake and Ladder problem

- **Input:**

- A board of size $N = 30$
- $\text{board}[i] = -1 \rightarrow$ normal cell
- $\text{board}[i] = j \rightarrow$ snake or ladder from i to j

- **Output:**

- Minimum number of dice throws required to reach the last cell (29)

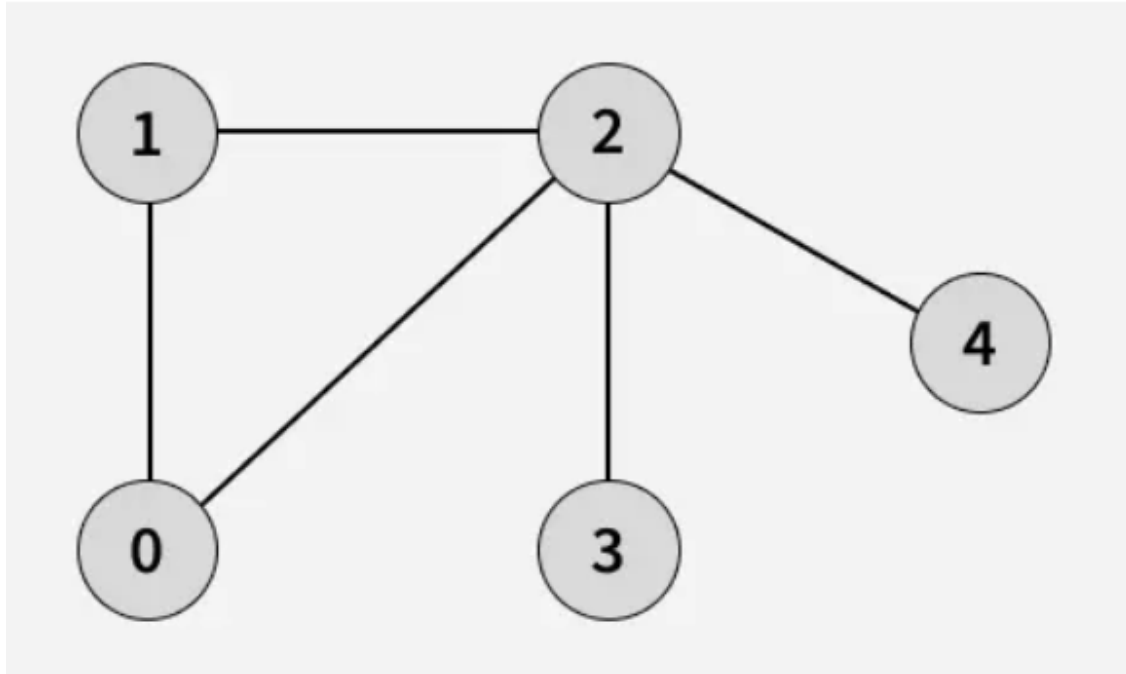


Example: Snake and Ladder problem

- **Explanation: The Following Are the Steps**
- The game starts from **cell 0** with **0 dice throws**.
A queue is used to explore all reachable cells level by level.
- From the current cell, all possible dice throws (**1 to 6**) are explored.
Each move represents **one dice throw**.
- If a cell contains a **ladder**, the player moves immediately to a higher cell.
If a cell contains a **snake**, the player moves down to a lower cell.
- Each visited cell is marked to avoid revisiting and infinite loops.
- The process continues until the **last cell (29)** is reached for the first time.
Because BFS is used, this guarantees the **minimum number of dice throws**.



Depth First Search



Depth First Search

- **Input:** $\text{adj}[][] = [[1, 2], [0, 2], [0, 1, 3, 4], [2], [2]]$
- **Output:** $[0, 1, 2, 3, 4]$

Explanation: The source vertex is 0. We visit it first, then we visit its adjacent.

Start at 0: Mark as visited. Print 0.

Move to 1: Mark as visited. Print 1.

Move to 2: Mark as visited. Print 2.

Move to 3: Mark as visited. Print 3.(backtrack to 2)

Move to 4: Mark as visited. Print 4(backtrack to 2, then backtrack to 1, then to 0).

- Note that there can be more than one DFS traversal of a Graph. For example, after 1, we may pick adjacent 2 instead of 0 and get a different DFS.



Example: Number of Islands

- **Input:** `grid[][] = [['W', 'L', 'L', 'L', 'W', 'W', 'W'], ['W', 'W', 'L', 'L', 'W', 'L', 'W']]`

Output: 2

W	L	L	L	W	W	W
W	W	L	L	W	L	W

